

Lab 4 - Cloud Data, Stat 215A, Fall 2025

November 19, 2025

1 Introduction

With the rapid pace of industrialization, global warming has become an increasingly urgent environmental concern over the past decade. Recent studies of environmental science research have been focused on the correlation between the rising temperatures and increasing carbon dioxide levels. Among the most conspicuous consequences of global warming are the changes occurring in the Arctic environment, including alterations in sea ice, water levels, and surface vapor pressure. However, due to similarities in scattering properties between ice and snow particles and cloud particles, distinguishing the clouds from the Arctic surface has always been a challenging task.

To monitor the global climate changes more accurately, NASA has launched the Multiangle Imaging SpectroRadiometer (MISR) technology. The cameras from MISR can take images from nine different angles, with four in the forward direction, one in the nadir direction and four in the aft direction [1]. With the help of images of these images from electromagnetic radiation, statistics researchers were able to develop algorithms to classify pixels from an image as either clouds or surfaces in the Arctic region. In the algorithm published in [1], three new features were constructed in addition to measurements from nine cameras. Then, thresholds were set on three added features to separate clear pixels. Finally, a QDA model was trained on the labels from the previous step.

In this lab report, we analyze 160 images from MISR data, along with three added features developed in published results. Among the 160 images, only three of them have expert labels which show whether a pixel represents clouds or the Arctic surface. Section 2 performs data cleaning to standardize the format of images by reshaping the size and scaling before training the data to a convolutional neural network to extract autoencoded features. We also do exploratory data analysis on relationship between images taken at various angles. Section 3 examines the correlation between features and does additional feature engineering to enlarge the input space. We introduce the gradient magnitude feature and the edge density feature, which capture edges for a given pixel and quantify the change in the intensity of radiance data across pixels.

In addition, we apply transfer learning techniques to learn features from unlabeled images and generated additional 20 autoencoded features. The modeling part uses the original data, two added physical features, and the autoencoded features to train models and predict results. Three models are built to classify pixels: Random forests (RF), Neural networks (NN), and SVM with a linear kernel. We adopt a new cross-validation scheme where we combine two images and separate them into nine folds to do cross-validation to tune hyperparameters. Then, we train the model with the tuned parameter on a third image. The random forest model outperforms the other three in terms of the AUC score, and the SVM with the linear kernel has the worst performance in all metrics except for the specificity value. Finally, we use the random forest and neural network models to study the stability and convergence pattern. We determine the feature importance through three different metrics and summarize the most important features from each metric. Post-hoc EDA is performed to take an in-depth look into the misclassification errors and assess their relationships with spatial information and features. Stability analysis is performed at the end by applying the same model on different validation images. The impact of our work is building new frameworks for cloud data classification and classification errors evaluation.

2 Data Cleaning and Exploratory Data Analysis

2.1 Data Cleaning

Before proceeding with any exploratory analysis, or any modeling, some basic data cleaning was required for the images in the dataset. First, the dataset was checked for any missing pixel values, however there were none. The data was presented in a fairly clean format with clearly labeled channels, expert features and labels provided with x,y coordinates for each image. However the images were of different sizes.

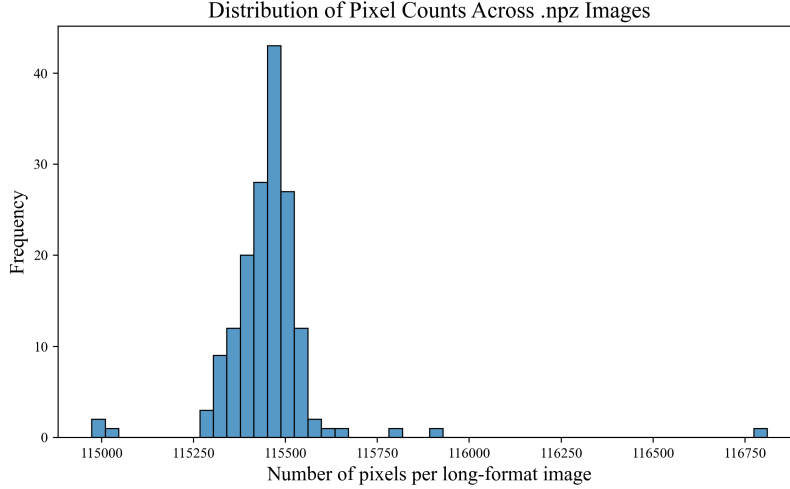


Figure 1: Distribution of image sizes by number of pixels.

In order to fix this, we proceed to standardize the format for each image. We first find the global bottom right-most point, i.e. the maximum values attained by the x and y coordinates over all images. We then "gridify" each image to be constrained to an 8-tensor (for each of the 8 channels) between the global minimum and maximum values. After filtering for any invalid coordinate values, we assign each element of the tensor the appropriate (x, y) pixel's channel intensity value. This reshapes each image to a consistent size and clean format while checking for inconsistencies.

For preprocessing before future modeling, we normalize the values of the intensity for each channel, i.e. constrain the mean to be 0 and variance to be 1. This is useful for models such as autoencoders which would give undue importance to certain unimportant features because of the imbalance in intensity units, or different patterns of intensity across each image.

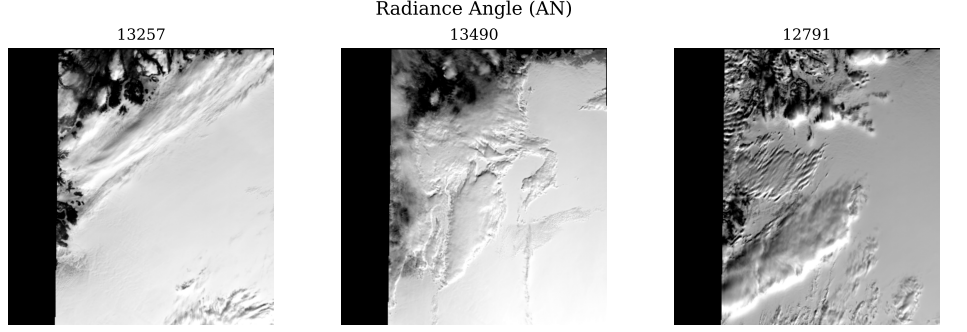
Finally, we also construct 9×9 patches for each pixel in each image, after padding the image with mirrored pixel values to ensure that pixels at the borders of the image also have meaningful patches. These patches are useful for constructing meaningful features and for models, such as our convolution autoencoder (AE), which aggregate information over such patches. Since patches bordering the left and right edges of the images had black borders, we made sure to exclude these patches during training of the autoencoder so as to not introduce any edge artifacts.

2.2 Splitting of data into train/test

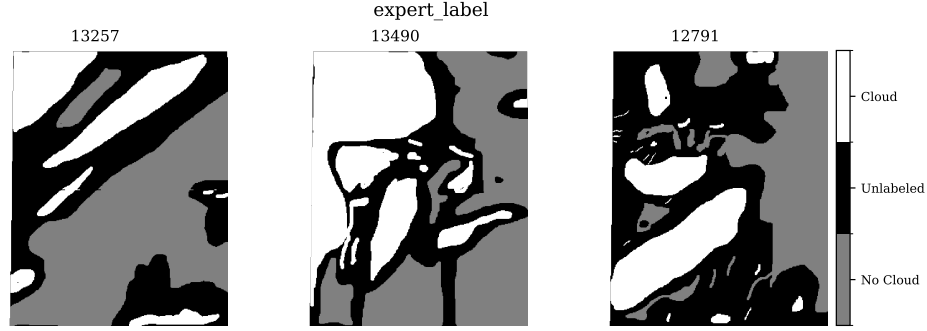
For our data exploration and cleaning, we simply use the 3 labelled images, and hence do not split our dataset into train/test. However, we use a more robust split for our autoencoder and modelling sections, which are described in great detail in subsequent sections 4 and 5.2.

2.3 Data Exploration

We proceed with the exploratory data analysis in three sections, analysing the different radiance angles and the relevance of the three expert features before splitting our data for training and testing. We limit our analysis to the three images with provided labels, i.e image numbers 13257, 13490 and 12791.



(a) Raw MISR red-band radiance from the nadir (An) camera.



(b) Expert-provided cloud labels for the same three scenes.

Figure 2: Visualization of the raw MISR input data and corresponding expert annotations.

2.3.1 Relationships between radiances

The Multi-angle Imaging SpectroRadiometer (MISR) instrument on board NASA’s Terra satellite captures imagery of the Earth’s surface from nine fixed viewing directions: four looking forward (Df, Cf, Bf, Af), one nadir camera (An), and four looking backward. Each camera records radiance in four spectral bands, enabling the characterization of surface anisotropy and cloud scattering properties across different view angles. The nadir-viewing AN camera observes the scene directly beneath the satellite and thus provides the most geometrically simple representation of surface brightness.

We explore the trends in these different radiance angles, and how differently they are distributed. We first include a visualization of these different angles for a reference image.

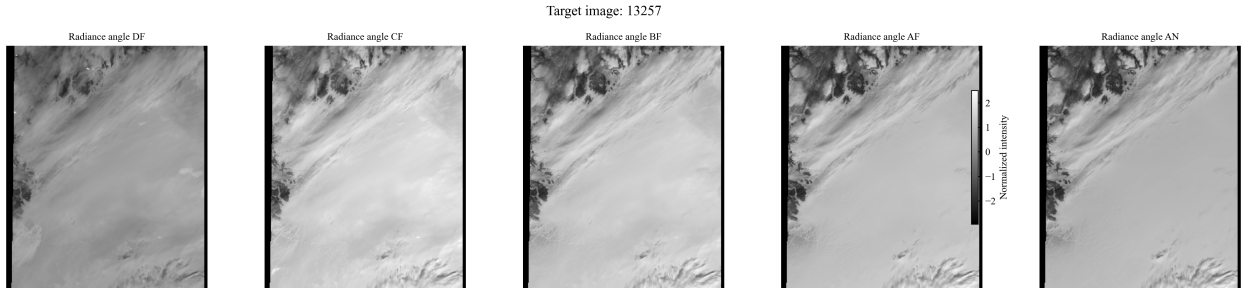


Figure 3: Radiance Angles for Img 13257.

We notice that while broad spatial trends remain the same, certain features in the top-left region of the image change across radiances. Different radiances also have different levels of sharpness, contrast and brightness. These trends are also consistent across the other two labelled images.

To further investigate these results we look at the distribution of these radiances, as plotted below.

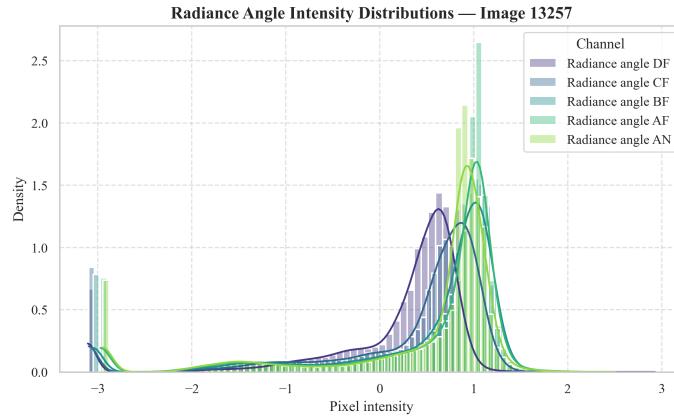


Figure 4: Radiance Angle Intensity Distributions for Img 13257.

We notice that the DF angle is visually clearly shifted leftward in comparison to the other radiance angles, and is hence dimmer. We also notice that the AN/AF angles are more highly peaked, indicative of higher contrast as borne out by the images above. These trends are again borne out by the other two labelled images as well.

Finally, we also look at the correlation of these different radiance angles to get a more quantitative measure of similarity. We use the spearman correlation coefficient as it better handles values that are not from symmetric Gaussian-like distributions non-parametrically.

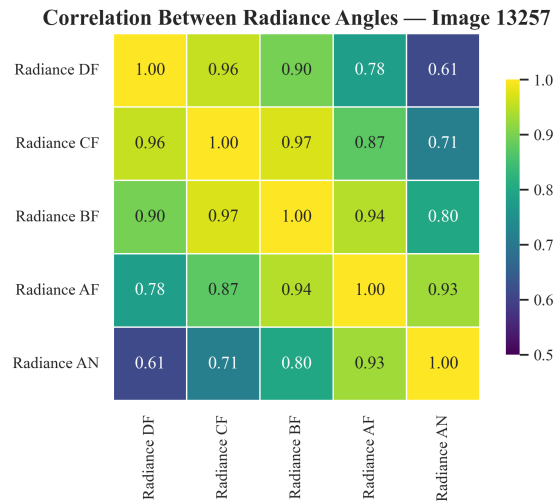


Figure 5: Spearman Rank Correlations for Radiance Angle Intensity - Img 13257.

This also bears out our previous results, with DF being significantly different from AN, and AN and AF containing similar information in terms of brightness and contrast. Further EDA and other analyses on the effectiveness of Expert Features such as NDAI, CORR and SD are included in the following section.

3 Feature selection and engineering

In this section, we analyze the available MISR-derived features to identify those that are most informative for distinguishing between cloudy and non-cloudy pixels. Using the expert-labeled data as ground truth, quantitative measures such as Fisher’s discriminant ratio and two-sample t -statistics are employed to assess feature separability, complemented by visualizations of class distributions. Building upon this analysis, new spatially aware features are engineered to capture additional structural information from the radiance data.

3.1 Features in the MISR Dataset

The MISR dataset [1] provides several radiance-derived features that were designed to separate cloudy from non-cloudy pixels based on the angular and spatial characteristics of reflected radiance. Three primary expert-engineered features are provided for each pixel: the *Normalized Difference Angular Index* (NDAI), the local *Standard Deviation* (SD), and the local *Correlation* (CORR).

Normalized Difference Angular Index (NDAI). The NDAI quantifies the difference in radiance measured between two MISR cameras observing the same ground location from different view angles—specifically, the forward-looking camera (DF) and the straight down-viewing camera (AN). The NDAI is defined as

$$\text{NDAI} = \frac{I_{\text{DF}} - I_{\text{AN}}}{I_{\text{DF}} + I_{\text{AN}}}, \quad (1)$$

where I_{DF} and I_{AN} denote the mean radiances averaged over a 4×4 group of 275 m pixels (corresponding to 1.1 km spatial resolution).

Standard Deviation (SD). The SD quantifies local spatial variability of radiance within a small neighborhood (patch) around each pixel. Cloudy regions often display higher local brightness variation due to cloud texture, shadowing, and optical thickness variability, while clear-sky surfaces (land or water) tend to be smoother, producing lower SD values.

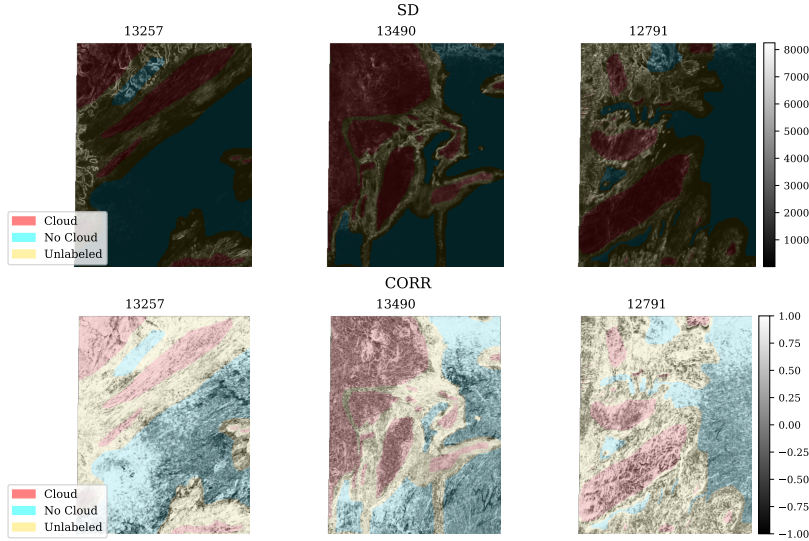


Figure 6: Spatial distribution of SD, and CORR values for the three labeled Images (IDs 13257, 13490, and 12791). (Top) SD values indicate that cloudy areas correspond to regions of high local variation with elevated SD values, whereas uniform surfaces such as ocean or land yield low SD values. (Bottom) CORR values show less pronounced differences between cloud and no-cloud regions compared to NDAI or SD, reflecting CORR’s lower discriminative power, though it still contributes useful complementary information about inter-angle reflectance behavior.

Correlation (CORR). The CORR feature measures the spatial correlation between radiance patterns observed from two different MISR viewing angles, typically between adjacent cameras.

Since surface reflectance tends to vary smoothly with view angle, clear-sky regions typically exhibit high correlation values, whereas clouds can appear less correlated across viewing directions. Accordingly, the visual difference between cloud and no-cloud regions in the CORR maps is subtle, as seen in Figure 6. Nevertheless, CORR provides complementary angular information that helps reduce ambiguity in boundary or mixed pixels when used in combination with the other features.

3.2 Feature Separability Analysis

To quantitatively assess the discriminative power of the available MISR features for cloud detection, we computed class-separability metrics using the expert-labeled pixels as reference. Specifically, we evaluated the t -value from a two-sample t -test between the *cloud* and *no-cloud* classes for each feature. The two-sample t -statistic was computed as

$$t = \frac{\mu_1 - \mu_0}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_0^2}{n_0}}}, \quad (2)$$

where μ_1 , μ_0 and s_1 , s_2 denote the class means and variances for the cloud and non-cloud samples, respectively. n_1 and n_2 are the sample sizes. High t values indicate features that exhibit strong class separation.

Figure 7 shows the computed t -values for all available features. The results confirm that the expert-engineered features NDAI and SD are the most informative predictors for distinguishing cloudy from clear-sky pixels. NDAI achieves the highest t -value, which is consistent with its theoretical design to capture angular anisotropy differences between isotropic cloud reflectance and anisotropic surface reflection. SD also exhibits strong separability, as cloudy regions tend to have higher local radiance variability. The low values for CORR are consistent with the findings of [1]. CORR exhibits almost no discriminative power the subset, likely because the scenes are spatially homogeneous and the correlation values vary little between cloud and surface pixels. This observation is consistent with [1]. Nevertheless, CORR is retained as an auxiliary feature in the literature because it improves spatial consistency when combined with NDAI and SD [1].

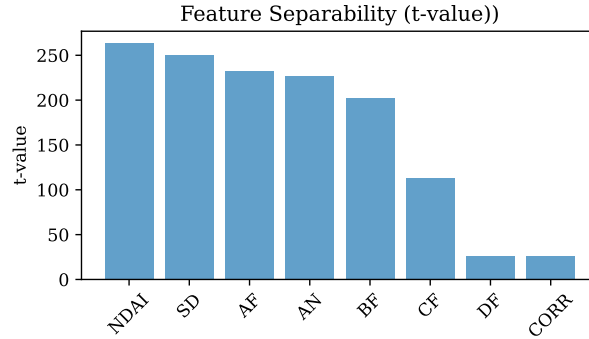


Figure 7: Feature separability metrics (t -value) computed between cloud and non-cloud classes using expert labels. Higher values indicate stronger discriminative power. NDAI and SD show the strongest separation, followed by CORR, confirming their suitability as core predictors for MISR-based cloud classification.

To further visualize the feature distributions across the three classes (cloud, no-cloud, unlabeled), Figure 8 presents the boxplots of NDAI, SD, CORR, and AN. NDAI shows a clear difference between cloud and non-cloud samples, although with a reverse trend relative to the definition of the original paper [1].

While the paper reports that clouds should exhibit NDAI values close to zero and surfaces large absolute values (due to stronger bidirectional reflectance), our implementation yields higher NDAI values for clouds. This inversion likely results from a sign convention in the angular difference formula ($I_F - I_B$ vs. $I_B - I_F$) but does not affect the separability of the classes. The SD feature, representing local texture, is consistently higher for clouds, reflecting their spatial variability and inhomogeneous structure. The CORR feature is inversely related to cloudiness, showing lower correlation values for clouds due to their volumetric scattering.

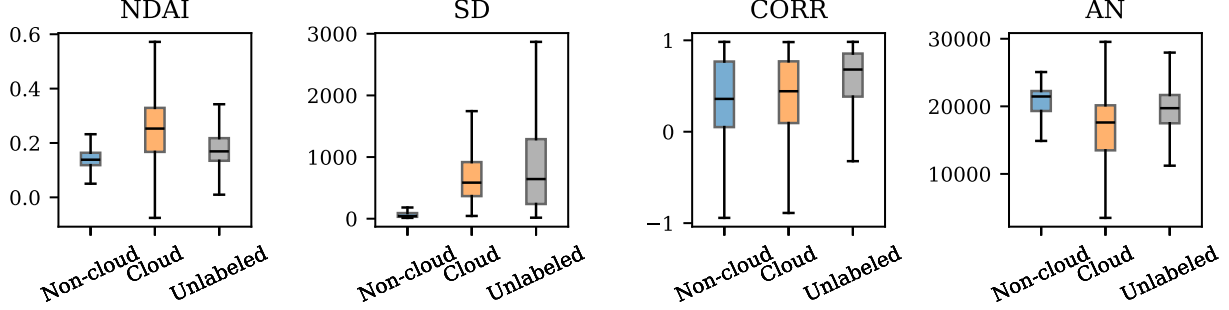


Figure 8: Boxplots of the main MISR features (NDAI, SD, CORR, and AN) for the three label classes (cloud, no-cloud, and unlabeled). NDAI and SD show clear separation between cloudy and clear-sky pixels, while CORR is inversely related to cloud presence. These trends are consistent across the three labeled image patches shown in Figures 2 and 6.

Finally, the AN radiance channel alone provides some brightness contrast but limited angular information. These observations are also consistent with that has been discussed in Section 3.1 and is shown in Figures 2, 6, 6, and 6.

3.3 Engineered Features

While the expert-designed physically informed and meaningful features (NDAI, SD, and CORR) provide valuable angular and local statistical information, they do not fully capture spatial structure or texture at a broader scale. To incorporate additional information from the local radiance patterns, two new spatially aware features were implemented: the Gradient Magnitude and the Edge Density. These features quantify the strength and spatial distribution of local intensity transitions within the radiance images and are derived using classical image-processing operators.

Gradient Magnitude. The gradient magnitude feature measures the local rate of change in radiance, highlighting edges and textured regions such as cloud boundaries. Given an image $I(x, y)$, the horizontal and vertical derivatives are computed using Sobel operators S_x and S_y [2]. The horizontal and vertical gradients are denoted as

$$G_x = I * S_x, \quad G_y = I * S_y,$$

where $*$ represents the convolution operation. The overall gradient magnitude is then given by

$$G(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}. \quad (3)$$

This quantity is high at locations of strong spatial variation (e.g., along cloud edges) and near zero in homogeneous regions such as clear-sky ocean surfaces. Since clouds often exhibit sharp local transitions in brightness, the gradient magnitude is sensitive to structural complexity.

Edge Density. While the gradient magnitude indicates where strong spatial transitions occur, it does not quantify how densely such transitions are distributed within a region. To capture this, the Edge Density feature measures the local fraction of pixels classified as edges within a sliding window. First, a global threshold T_p is computed as the p -th percentile of all gradient magnitudes in the image. A binary edge mask is then defined as

$$E(x, y) = \begin{cases} 1, & \text{if } G(x, y) \geq T_p, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

The edge density $D(x, y)$ is obtained by averaging this binary edge mask over a local neighborhood W of size $n \times n$ pixels:

$$D(x, y) = \frac{1}{|W|} \sum_{(i, j) \in W} E(x + i, y + j), \quad (5)$$

where $|W| = n^2$ is the number of pixels in the window. $D(x, y)$ represents the proportion of edge pixels in the local neighborhood, ranging from 0 (no edges) to 1 (dense edge region). Regions with high $D(x, y)$ correspond to structurally complex cloud areas, while low values indicate smooth and homogeneous surfaces.

Figure 10 illustrates the gradient magnitude and edge density. The gradient magnitude measures local intensity changes and highlights edges and transitions in brightness, while edge density quantifies the proportion of strong edges within a local neighborhood. Conceptually and also from the produced pattern, edge density and SD are similar. When each feature is used individually as a simple classifier, the AUC and t-value indicate its discriminative strength between cloudy and clear pixels. The results show that edge density achieves the highest t-value (344) and a strong AUC of 0.90, and thereby outperforms all other features in terms of the t-value, including physical ones such as NDAI ($t = 264$, $AUC = 0.82$) and SD ($t = 250$, $AUC = 0.94$). The large t-value suggests a clear and consistent separation of the two classes, especially in identifying clear sky areas, since clouds often have more texture and edges than clear sky. The gradient magnitude also performs well ($t = 192$, $AUC = 0.93$) and confirms that spatial texture information contributes meaningfully to distinguishing clouds.

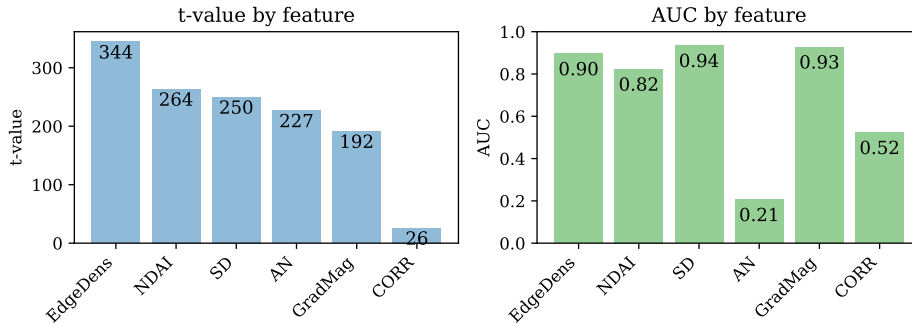


Figure 9: Comparison of feature performance using AUC and t-values for individual single-feature classifiers. The edge-based features (Gradient Magnitude and Edge Density) achieve AUC values comparable to or exceeding the traditional physical features, suggesting that spatial texture information is useful for cloud detection.

4 Learning features with an autoencoder

We developed a fully convolutional autoencoder (AE) which had three $\times$ (convolution, batch norm, ReLU) layers. We used 80% of the unlabeled images for training and the rest for computing the validation loss. To ensure that the AEs had high predictability, we trained them for 20 epochs (about 1 hour runtime on Bridges 2), at which point the training and validation loss were very stable and hardly moving. Training and validation losses were comparable to each other, indicating that the models were generally not overfitting. Then, we fine-tuned these models on the labeled images, without supplying the labels. The learning curve for fine-tuning was much faster, so this was performed for 10 epochs instead of 20 (<5 minutes runtime).

The design of all of our AEs was similar in that their latent dimension was the same as the image patch. This constraint was motivated by the fact that we would need to use these embeddings for downstream classification tasks. Thus, we need to be able to assign the AE features to a single pixel. Our first versions had flattened latent dimensions, but these were challenging to use for prediction because each embedding corresponds to a patch of 9×9 pixels. This prompted us to use $N_{channel} \times \text{height} \times \text{width}$ embeddings, in which each channel in the embedding corresponds unambiguously to a single pixel. We chose $N_{channel} = 20$, although this was a judgement call based on limited testing. In figure 11, we show training and loss curves

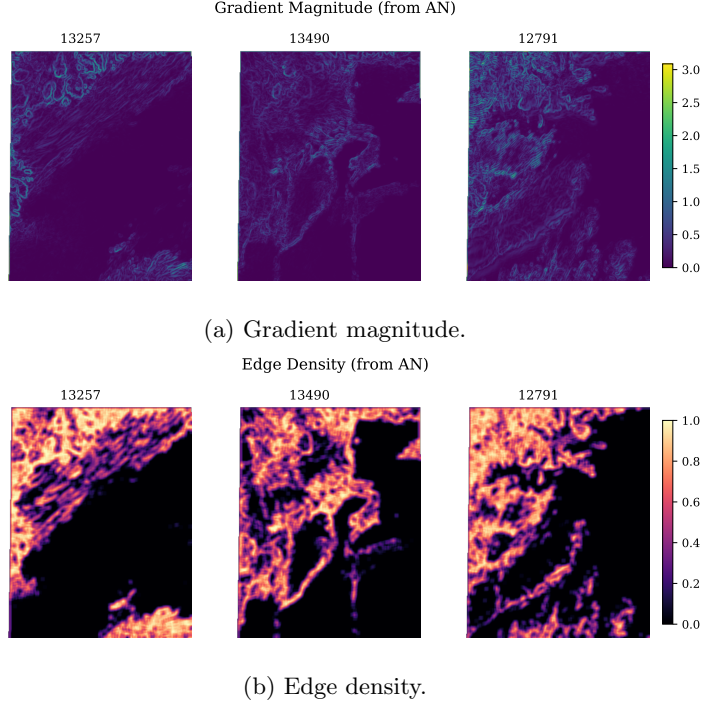


Figure 10: Computed (a) Gradient magnitude and (b) Edge density. Clear sky shows less edges in general, which makes the edge density a good predictor for clear sky, since clouds often show higher contrasts and more edges.

for the selected AE, and some interesting AE features for image 13257. The AE features extract various contours, textures, and luminosity patterns from the image. See section 5.4.3 for an analysis on the predictive potential of these embeddings.

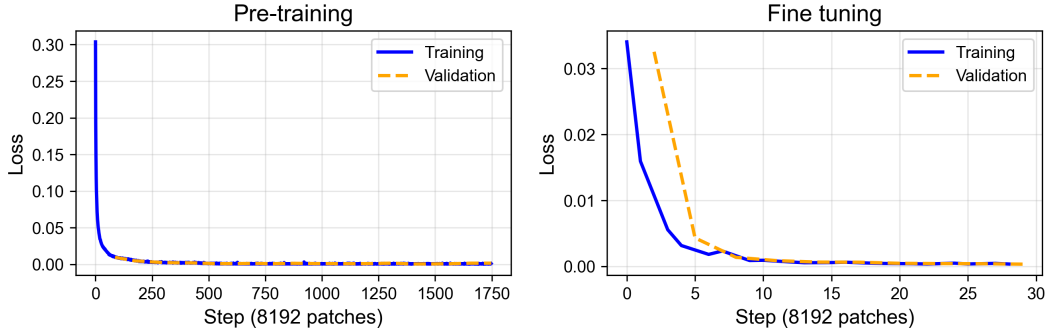


Figure 11: Top) Training and validation loss curves for 20 epochs of pre-training (left) and fine-tuning (right). Pretraining use 80% of the unlabeled images, fine tuned on 3 labeled images.

5 Modeling

5.1 Classifiers Used

Below we include a brief description of the three modelling approaches we test for pixel classification. These include a broad mathematical description, assumptions required and their verification, as well as a discussion on their advantages and disadvantages.

5.1.1 Random Forests

Random Forests are an ensemble method for classification that aggregates information over multiple Decision Tree models. For each individual tree, a random subsample of the data is selected (bootstrapped) to train on, and a random subset of the features is selected at each split. The trees are then fit using the usual CART procedure. This results in an improvement over bagging and helps decorrelate the individual trees, and also has the useful feature of OOB (Out of Bag) estimates. The final prediction class provided by the model is the one that occurs in the most number of trees.

A central assumption of the Random Forest model is *IID* data, i.e. independent and identically generated. This is necessary while bootstrapping the data into individual trees and for consistency guarantees. However, the pixels in our image are clearly spatially correlated with their neighbours. To reduce this bias, we cross-validate by splitting data by larger blocks into train/test rather than at the pixel level. We also train a Spatial Random Forest model that incorporates lag terms for local neighbours.

Other assumptions of the model include smoothness of $E[Y|X]$, a stable conditional distribution $P(Y|X)$, which are important for mathematical guarantees, but are more difficult to check empirically and are usually true. Finally, we also require that the features have finite variance and are not perfectly correlated which we empirically verify.

5.1.2 Neural Network

The Neural Network model is a compact feedforward neural network designed to classify individual pixels using only their local feature vectors. Its architecture consists of an input layer (number of features), followed by a fully connected hidden layer with 256 neurons. To improve training stability, the hidden layer is regularized using Batch Normalization, and a ReLU activation introduces nonlinearity. A Dropout layer (with dropout probability $p = 0.2$) is applied to avoid overfitting. The dropout randomly deactivates neurons during training. The final fully connected layer outputs a single logit, which represents a probability that can be converted into a cloud-vs.-clear probability using a sigmoid function or simple threshold. This model assumes that each pixel can be classified independently based on its own spectral and angular features, without incorporating any spatial context from neighboring pixels.

Before training, the input features were z score normalized to ensure that each feature contributes on a comparable scale. The normalization is essential for stable neural-network in the gradient calculations and prevents the model from being biased toward features with larger numerical ranges.

For neural network models to perform well, it is assumed that the relationship between the input features and the cloud labels is smooth and learnable, that the features are properly scaled, and that the training data are representative of the overall distribution.

In this case, these assumptions are satisfied, as the physical features (e.g., NDAI, SD, CORR, and radiance) are continuous, normalized, and derived from physically meaningful quantities, and the labeled images capture typical cloud and clear-sky conditions with some spatial coherence between neighboring pixels. The same holds for the autoencoder-derived features, since the autoencoder is trained to learn smooth, low-dimensional representations of the same input.

5.1.3 Support Vector Machines (SVM)

The support vector machine (SVM) is another useful approach to separating observations into two classes. The goal of the SVM algorithm is to find a hyperplane that can maximize the margins, which represent the distances between the data points and the hyperplane. The linear separable SVM can be defined as [3]:

$$f(x) = \beta_0 + x^T \beta \tag{6}$$

The hyperplane that separates the two classes corresponds to the case where $f(x) = 0$. A given observation x_i is classified into Class 1 when $f(x_i) > 0$, and Class -1 otherwise. In addition to Linear SVM, SVM with the radial basis kernel (RBF) is one common technique that transforms the original input space. The advantage of RBF is that it creates non-linear boundaries between data to separate classes that are not easily separable by a linear hyperplane. For observations that form clusters that are not well separated, SVM with the RBF kernel can more effectively distinguish two different classes.

When training the model with SVC module in the sklearn package, we mainly tuned two hyperparameters: 'C' and 'kernel'. The parameter 'C' controls the strength of regularization by introducing a penalty for misclassified samples. A higher C value assigns a higher penalty to misclassification errors, which allows the model to fit the data more closely and reduce training errors. The 'kernel' parameter specifies the exact aforementioned kernel method used. Firstly, we select a grid of 'C' values of [0.5, 1, 1.5] and 'kernel' values as 'linear' and 'rbf' when doing the hyperparameter tuning with the cross-validation method introduced in the subsequent subsection. However, since the number of pixel values exceeds 70,000 for only one image, our model training step has a large computational cost when both linear and rbf kernels are used. We decide to proceed with only the linear kernel method to select an optimized value for the 'C' parameter and do model training on the training set.

The major assumption of using linear SVM techniques is that the data can be linearly separable with a hyperplane. To test this assumption, we make scatterplots between different variables and color the data points based on labeled classes in each graph. Unfortunately, it seems that there is not a clear linear boundary line between two classes because the data points of both colors are clustered together in the majority of pairplots. The only exceptions exist in the scatterplots that contain the feature CORR and the encoded feature 12 from the autoencoder. From the scatterplots of these two variables against other variables, we might be able to draw a linear plane that separates classes of cloud vs no cloud. Therefore, the linear separability assumption of the linear SVM model is only partially satisfied in our data.

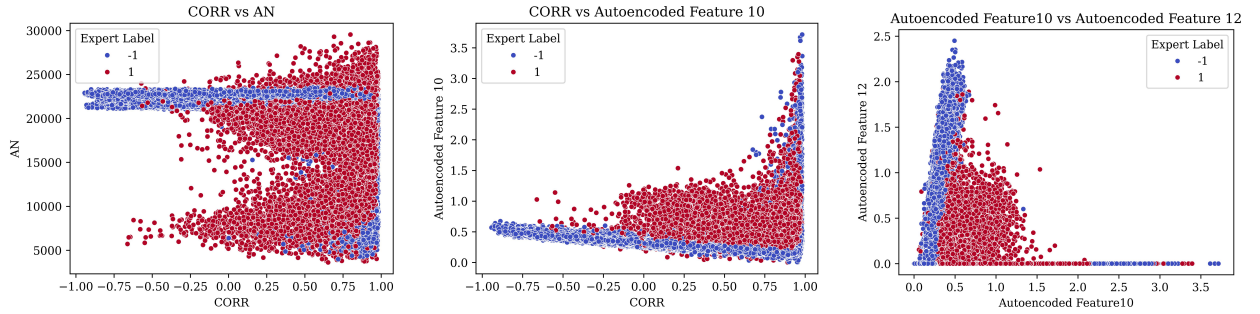


Figure 12: Scatterplots between features to test linear separability assumptions of SVM (a) Physical Feature CORR vs Feature AN (b) Physical Feature CORR vs Autoencoded Feature 10 (c) Autoencoded Feature 10 vs Autoencoded Feature 12

Another assumption of the SVM model is that the observations follow the IID rule: independent and identically generated. Therefore, we split the images into folds and do training and testing on each fold as described in the next section to reduce the correlation between pixel values. Finally, because SVM is sensitive to outliers, and we verify that the assumption held in our data. From the scatterplots between different variables, we do not see significant outliers that could potentially cause a misfit of the model.

5.2 Cross-validation scheme

The nature of our data necessitates the use of a robust cross validation and train/test/validation split over the data. The pixels within each image can be very strongly correlated with its neighbours and have very similar features. There may be other very strong spatial correlation trends within our data. Thus, simply using random splits for training and testing our models may falsely overestimate their accuracy, as neighbouring pixels may be present in both splits leading to data leakage into our test dataset.

Indeed, this pattern is confirmed through a rudimentary analysis using a vanilla Random Forest with default

hyperparameters and different training pipelines. We used a fully random (pixel-level), random at the spatial 64×64 block level, as well as image-level splits for cross-validation and evaluated the same model trained on each of these splits. We maintained an 80 : 20 split for the first two approaches and a 66 : 33 split as necessitated by image-level CV.

Metric	Pixel-level CV	Block-level CV	Image-level CV
Accuracy	0.9680	0.9310	0.7981
AUC	0.9955	0.9639	0.9120
Sensitivity	0.9702	0.9387	0.9233
Specificity	0.9668	0.9043	0.7223
Brier score	0.0239	0.0559	0.1380

Table 1: Cross-validated performance across pixel-, block-, and image-level CV

As expected, the model performs seems to be significantly better (96.8% accuracy) when using a fully random split. However the performance of the same model on the same dataset seems to dip when using block-level (93.1% accuracy) and image-level (79.8% accuracy) splits. This clearly evidences the data leakage from train to test datasets.

In an effort to mitigate this bias, we implement a more careful train/test/validation split for our three images with labeled data. We first separate an image (No. 13257) for validation out of the three. We then proceed to split the other two images into 9 i.e. 3×3 blocks with roughly an equal number of pixels in each. For each fold, we assign 1 of these 9 blocks from both images as test, and after removing a 2-pixel wide padding around the test block, include the rest of both images. We outline this process in the diagram below.

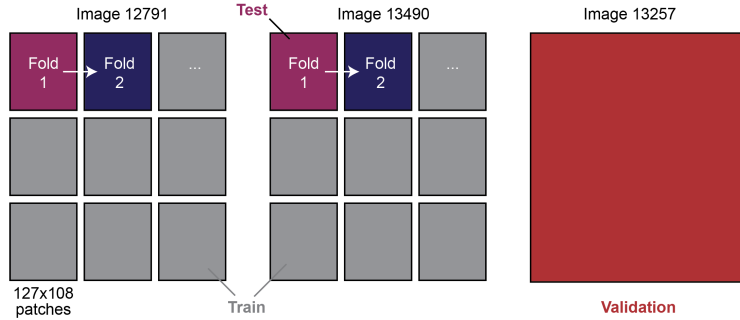


Figure 13: Cross-validation scheme for training, testing, and validating the three models of interest.

This yields a 9-fold cross validation split into train/test, while reserving a third unseen image for final validation. The large block size as well as the padding between train and test data minimize the spatial correlations and information leakage described above. In the subsequent sections, the train/test folds are used to inform model architecture and tune hyperparameters, before validating on the final image.

5.3 Assessing Model Fit

Figure 14 summarizes the performance of our models across accuracy, AUC, sensitivity, specificity, and Brier score ordered from best on the left to worst on the right for each subplot. Overall, the Random Forest and the neural network (NN) trained on all available features (10 measured and physically informed features and 20 latent embeddings from the Autoencoder) achieve the strongest and most balanced performance across the metrics, whereas the support vector machine (SVM) performs slightly worse, likely due to the linear formulation we used and therefore limited ability to capture nonlinear trends in the data.

A trade-off is the balance between sensitivity (correctly identifying cloud pixels) and specificity (correctly identifying clear pixels). The results show that the neural network achieves higher sensitivity than the Random Forest and SVM, meaning it detects a larger fraction of true cloud pixels. In contrast, the Random Forest achieves higher specificity, making it more conservative and less likely to produce false cloud classifications. However, the NN’s specificity is much closer to the Random Forest’s specificity than the Random

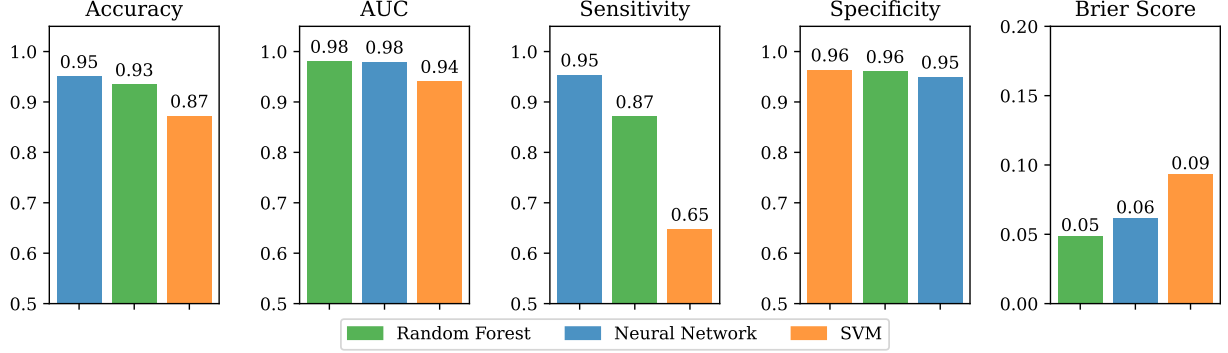


Figure 14: Model Fit. Models are ordered from best (left) to worst (right) within each metric. The Random Forest and the neural network (NN) trained on all features achieve the strongest overall performance, while the support vector machine (SVM) performs slightly worse due to its linear nature.

Forest’s sensitivity is to the NN’s sensitivity, which indicates that the NN offers a more balanced detection profile overall. Beyond this trade-off, the two neural network variants perform very similarly across all other metrics, while the SVM lags slightly behind both methods. This lower performance is likely due to the relatively simple SVM setup and the use of a linear rather than a nonlinear kernel, which limits its ability to capture the underlying structure in the features.

Across the remaining metrics (accuracy, AUC, and Brier score), the Random Forest and the neural network shows very similar and strong performance, while the SVM falls behind. The weaker SVM performance is expected due to the fact that we only trained a linear SVM, which is less suited to modeling nonlinear structures.

Interestingly and although not shown in the figure, the NN trained only on the five physically motivated features (NDAI, CORR, SD, AN, and Edge Density) performs almost as well as the NN using all features, which confirms the power and importance of a domain-driven feature set.

Overall, the results highlight the robustness of both tree-based ensemble methods and neural networks for pixel-wise cloud detection. For the post-hoc exploratory data analysis, we therefore focus on both the neural network and the Random Forest, as these two models deliver the strongest and most balanced performance profiles.

5.4 Diagnostics and Feature Importance

We further analyze our Random Forest model to understand its stability and convergence to analyze the stability of our predictions. We choose the Random Forest model due to its superior performance. We also investigate feature importance.

5.4.1 Stability and Convergence

For diagnostics with respect to our Random Forest model, we focus on convergence with respect to its size, i.e. the number of trees grown within the forest. This is reflected by the *n_estimators* hyperparameter. To illustrate convergence patterns, we look at the probability that a pixel i was assigned to cloud, and look at how this differs with a larger number of trees.

More specifically, if $\hat{p}_n(X_i)$ denotes the predicted class-1 probability for observation X_i produced by a random forest with n trees. When the forest grows from $n - 1$ to n trees, the change in predicted probability for observation i is

$$\Delta_i(n) = |\hat{p}_n(X_i) - \hat{p}_{n-1}(X_i)|.$$

The average prediction instability at forest size n over a dataset $\{X_1, \dots, X_N\}$ is

$$I(n) = \frac{1}{N} \sum_{i=1}^N |\hat{p}_n(X_i) - \hat{p}_{n-1}(X_i)|.$$

Since the random forest only uses a subsample of the data to train each tree, we can also use the Out-Of-Bag or OOB estimates as a validation metric for the same within the training data. This generates the probability for pixel i using only those trees that were not trained on i . Below, we plot this quantity separately for out-of-bag (OOB) predictions and for our external validation set.

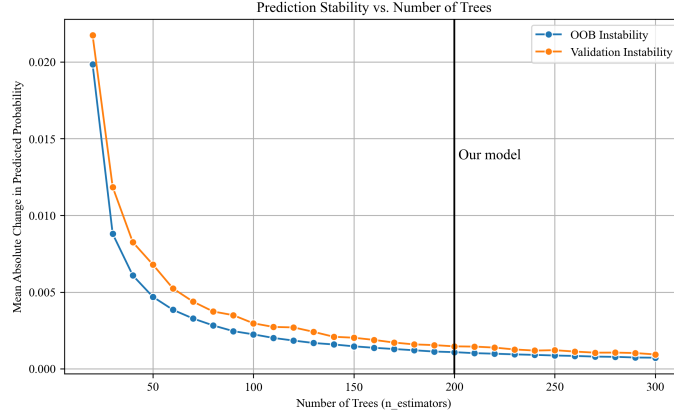


Figure 15: Convergence of Random Forest.

We see that the predicted probabilities converge (The successive differences flatten out close to 0) after approximately $n = 150$, and hence our choice of $n = 200$ is validated, and our predictions are stable.

To further assess stability, we also investigate the variance of these predicted probabilities among the trees within our final RF model. Similar to above, we analyse this for both OOB estimates and Validation estimates, after separating by class.

More mathematically, let $\mathcal{T}(i)$ denote the set of trees for which sample i is OOB, and let $|\mathcal{T}(i)|$ be its size. The per-sample OOB prediction variance (variability) is

$$\text{Var}_i = \frac{1}{|\mathcal{T}(i)|} \sum_{t \in \mathcal{T}(i)} \left(\hat{p}_i^{(t)} - \bar{p}_i \right)^2.$$

Similarly, The per-sample validation prediction variance (validation variability) is

$$\text{Var}_i^{\text{val}} = \frac{1}{K} \sum_{k=1}^K \left(\hat{p}_i^{(n_k)} - \bar{p}_i^{\text{val}} \right)^2.$$

We look at the distribution of these variances across pixels within our training and validation sets respectively. We also stratify by class to identify trends.

We notice that though the validation variability is higher on average than OOB variability, the vast majority of both are very highly concentrated near 0, indicating little to no variance in our model's estimates for the majority of the pixels. However, cloudy pixels have higher variability on average than clear ones particularly in the validation set. This may be due to the class imbalance and fewer number of cloudy pixels in our training data, meaning our model finds it harder to generalize these patterns.

However, the consistently low value of variability, coupled with the stability with $n_estimators$ shown earlier, give us confidence that our model is largely stable and gives good predictions.

In addition to the Random Forest, we also examine the convergence behavior of the neural network. Figure 17 shows that the training loss decreases smoothly, while the validation loss follows a similar trend without

Histogram of Per-Pixel Variance by Expert Class (Clear vs Cloudy)

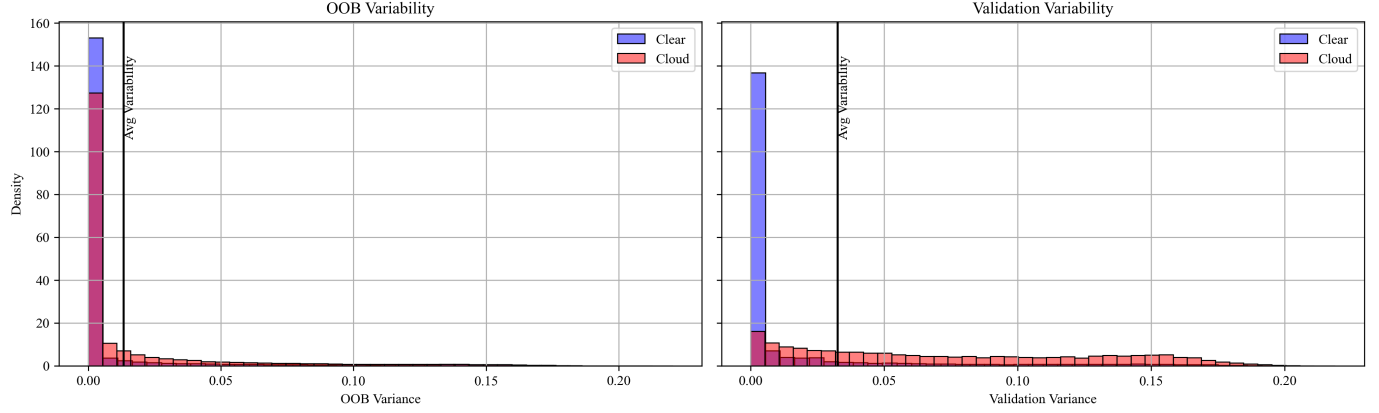


Figure 16: Per-Pixel Variability of the Random Forest.

large fluctuations. This indicates that the model learns a stable decision boundary without severe overfitting. Although the validation loss plateaus at a higher value than the training loss as expected for a moderately expressive model trained on noisy, real-world labels the overall stability of both curves lends confidence that the neural network converges reliably and generalizes well to unseen regions of the images.

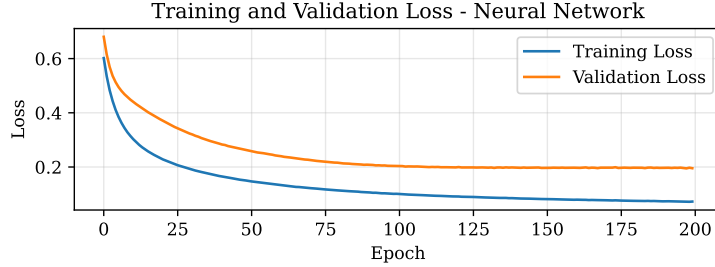


Figure 17: Training and validation loss curves of the neural network.

5.4.2 Feature Importance

We analyze feature importance with respect to the final Random Forest model, and calculate MDI+ and permutation-based feature importance scores for our model to better understand it's key components. We also include permutation-based metric on OOB predictions for a more accurate comparison with MDI+ as both are based on the training data, while the usual permutation-based importances are based on the validation data. The metrics are briefly described below.

MDI and MDI+. For a tree node t that splits on feature j , the impurity decrease is

$$\Delta i(t, j) = i(t) - p_L i(t_L) - p_R i(t_R), \quad p(t) = \frac{n_t}{n},$$

and the corrected impurity-based importance is

$$\text{MDI}_j^+ = \frac{1}{T} \sum_{k=1}^T \sum_{t \in \text{splits on } j} p_k(t) \Delta i_k(t, j).$$

Notation: n_t is the number of samples reaching node t , $i(t)$ is node impurity, $p_k(t)$ is the proportion of samples reaching t in tree k , and T is the number of trees.

Permutation Importance (OOB). Let $L(f; \cdot)$ be a loss and $X_{\text{OOB}}^{\pi_k(j)}$ the OOB samples of tree k with feature j permuted. The OOB permutation importance is

$$\text{PI}_j^{\text{OOB}} = \frac{1}{T} \sum_{k=1}^T \left[L\left(f_k; X_{\text{OOB}}^{\pi_k(j)}, y_{\text{OOB}}\right) - L(f_k; X_{\text{OOB}}, y_{\text{OOB}}) \right].$$

Notation: f_k is tree k , X_{OOB} and y_{OOB} are its out-of-bag samples, and $\pi_k(j)$ is a permutation applied only to column j for tree k .

Permutation Importance (Validation). For a validation set $(X_{\text{val}}, y_{\text{val}})$ and a single permutation $\pi(j)$ of feature j ,

$$\text{PI}_j^{\text{val}} = L\left(f; X_{\text{val}}^{\pi(j)}, y_{\text{val}}\right) - L(f; X_{\text{val}}, y_{\text{val}}).$$

Notation: f is the trained forest, L is any chosen loss, and $\pi(j)$ is a random permutation applied to feature j over the entire validation set.

We then identify the 10 most important features according to each and list them below.

MDI+ (Top 10)	OOB Permutation (Top 10)	Validation Permutation (Top 10)
Radiance angle AN (0.1102)	SD (0.017603)	Radiance angle AN (0.068108)
Gradient Magnitude (0.0594)	Gradient Magnitude (0.008177)	Gradient Magnitude (0.040998)
SD (0.0512)	AutoEncoder 16 (0.002068)	SD (0.042102)
AutoEncoder 17 (0.0468)	Edge Density (0.001461)	Edge Density (0.028211)
CORR (0.0322)	AutoEncoder 8 (0.000957)	AutoEncoder 17 (0.006617)
AutoEncoder 16 (0.0170)	AutoEncoder 18 (0.000665)	AutoEncoder 16 (0.003441)
NDAI (0.0078)	CORR (0.000468)	AutoEncoder 8 (0.001879)
Edge Density (0.0054)	Radiance angle BF (0.000358)	AutoEncoder 18 (0.000894)
AutoEncoder 20 (0.0048)	NDAI (0.000329)	AutoEncoder 9 (0.000718)
AutoEncoder 5 (0.0042)	AutoEncoder 19 (0.000314)	AutoEncoder 20 (0.000492)

Table 2: Top 10 most important features under MDI+, OOB permutation importance, and validation permutation importance. Features appearing in **all three methods** are highlighted in bold.

The MDI based features are often biased towards features with higher cardinality, and therefore we include permutation based features as well. In the spirit of the PCS framework and the approach towards combining multiple algorithms we compare results across all 3 methods. We see that both our created features, Gradient Magnitude and Edge Density are important across all 3 metrics. Several of the latter Autoencoder features from 15 – 20 are also important across all 3. SD seems to play a very important role as well, which confirms the results from EDA and section 3.3. The unchanged AN intensity also seems to be the most important feature across MDI+ and Validation Permutation, suggesting that it captures a lot of information beyond these additional features as well.

To complete the feature-importance analysis and investigate robustness across modeling approaches, we also computed permutation importance for the neural network. Because neural networks do not provide built-in feature importance scores, permutation importance offers a model-agnostic way to quantify how much each feature contributes to predictive performance. For each feature, we independently permuted its values in the validation set while keeping all other features fixed and measured the resulting drop in AUC. Larger drops indicate more important features. As shown in Figure 18, AN, CORR and the two engineered features - Gradient Magnitude and Edge Density - emerge as the most influential inputs for the neural network, followed by a mix of the physically motivated features such as SD, and CORR and several autoencoder embeddings, though with smaller importance scores. This aligns with earlier EDA and model-comparison

results: spatially informed texture features provide strong predictive signal, while the high-dimensional autoencoder features contribute marginal additional value. Overall, the permutation-based results support the conclusion that carefully engineered physical features dominate the NN’s predictive behavior, but latent embeddings can add important predictive value.

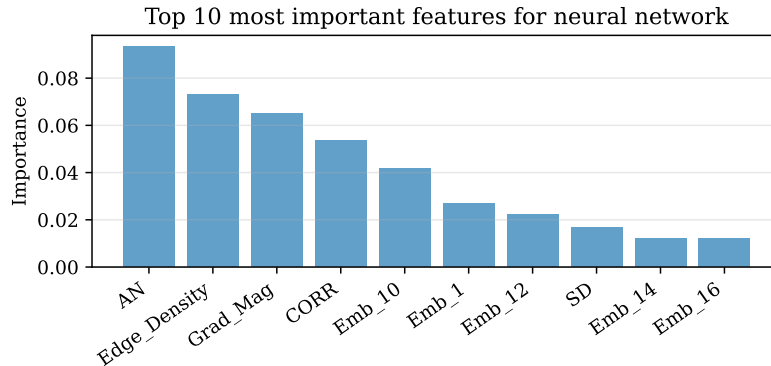


Figure 18: Top 10 most important features for the NN based on permutation importance from the validation set. AN and engineered features show the largest impact on model performance, followed by several autoencoded features

5.4.3 Predictability of autoencoder features

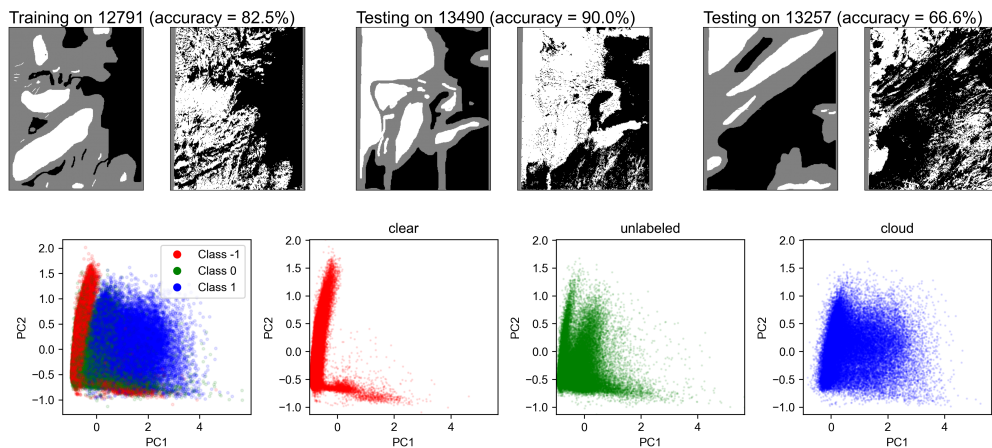


Figure 19: (Top) L1-penalized logistic regression trained on autoencoder embeddings for image 12791 and tested on the other two labeled images. The training transfers well for one image (13490) but poorly for the other (13257). (Bottom) PCA of autoencoder embeddings for image 13490. Notice that just two dimensions in the embeddings can discriminate the clear and cloudy pixels relatively well.

We used L1-penalized logistic regression to determine if the AE embeddings had any predictive potential on their own. We trained on image 12791, and tested on images 13490 and 13257. The training accuracy was 82.5% and the test accuracy was 90% and 66% respectively (Figure 19, top). In other words, the AE embeddings had the potential to be quite good at predicting whether a pixel was clear or cloudy on an image it had never seen, but the embedding thresholds may not generalize well across images. This procedure was purely to confirm that the AE embeddings retained some information that could be useful for prediction. Another way we confirmed this was by performing PCA on the AE embeddings. We find that the first two PCs of the autoencoder embeddings could separate cloudy and clear pixels decently well (Figure 19, bottom). Interestingly, although we added an L1 penalty, none of the 20 autoencoder features were shrunk to zero, suggesting that they were all somewhat useful for prediction. For this reason, we supplied all 20 autoencoder embeddings as features for downstream modeling.

5.5 Post-Hoc EDA

In this subsection, we perform post-hoc exploratory data analysis of the random forest model to visualize the distribution of misclassification errors on the image 13527. We first plot the true expert labels and soft probabilities to see which areas of the image are most likely to be misclassified.

From Figure 20, the large continuous area at the bottom right portion of the image is mostly correctly classified as “No Cloud” with a few misclassified pixels at the edge of the same area. The ‘Cloud’ regions at the bottom and the left of the image are also correctly labeled. However, one area at the top left corner is mostly incorrectly identified as ‘Cloud’.

To further quantitatively analyze the misclassification errors, we plot a heat map of brier score on the image to measure the accuracy of probabilistic distributions. The brier score describes the squared differences between the predicted probabilities and the actual outcome. We notice that the ‘island-like’ regions at the top left of the image have consistent high brier scores across pixels, with some pixels even exceeding 0.5. The heatmap results align with the earlier observation of high rate of false results in the same regions.

Because the main objective of the predictive modeling is to correctly identify the pixels labeled as ‘cloud’, we would like to improve the sensitivity values ($TP / (TP + FN)$) of prediction while maintaining accuracy of the model. To properly select a threshold value for prediction, we compute the change in sensitivity score, accuracy score, and specificity score with respect to the threshold values. Based on the plot, the threshold value of 0.3 appears to improve the sensitivity and keep a relatively high accuracy score compared to the threshold value of 0.5.

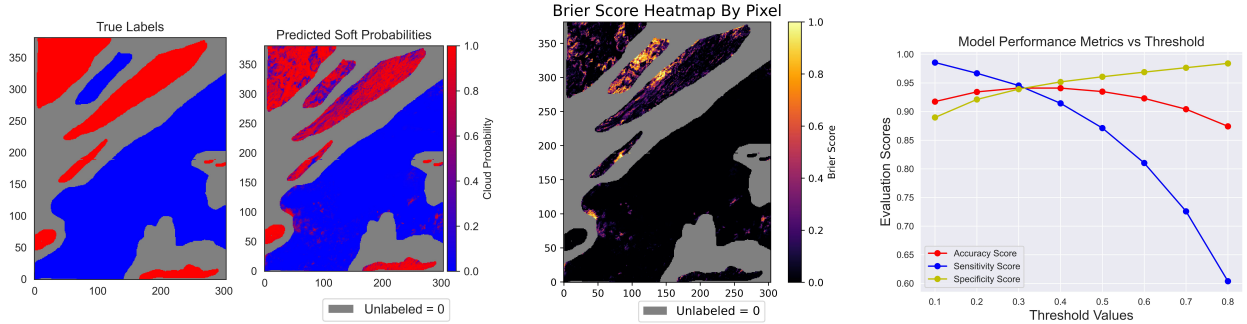


Figure 20: Image 13527 (a) Side-by-side plot of true labels and predicted probabilities of clouds (b) Brier score heatmap by pixel (c) Accuracy, sensitivity, and specificity change with different thresholds

The results indicate that we increase the number of pixels that are correctly labeled as ‘Cloud’ classes in the top left region of the image by lowering the threshold values. However, the second ‘isolated island’ in the same region has a higher number of false positives. We suspect that the region with ‘No Cloud’ labels has similar values to the neighboring “islands” with “Cloud” labels in certain features and causes the model to misclassify the majority of pixel values. To investigate the influence of features on the misclassification, we subset the data by limiting the x and y coordinates to the top left region of the image ($x \leq 150$ and $250 \leq y \leq 370$), and do boxplots and scatterplots on selected features to show the distribution of values.

After filtering the data based on x and y coordinates within the top left corner, we compute the t-statistics of each feature between two classes, select the five features with highest absolute values of t-statistics and make boxplots on them. The figures show that the No Cloud and Cloud classes exhibit largely similar value distributions across the features, with no strong separation between the two classes. Even the scatterplots of the features with the highest absolute t-statistics (feature CORR and autoencoded feature 16) show substantial overlap between datapoints. The p-value is not a useful metric here due to the large sample size. The visualization of feature value distribution explains the high rate of false negative classification of pixels on the ‘second island’ from the top left corner of the image as the feature values resemble those from neighboring islands.

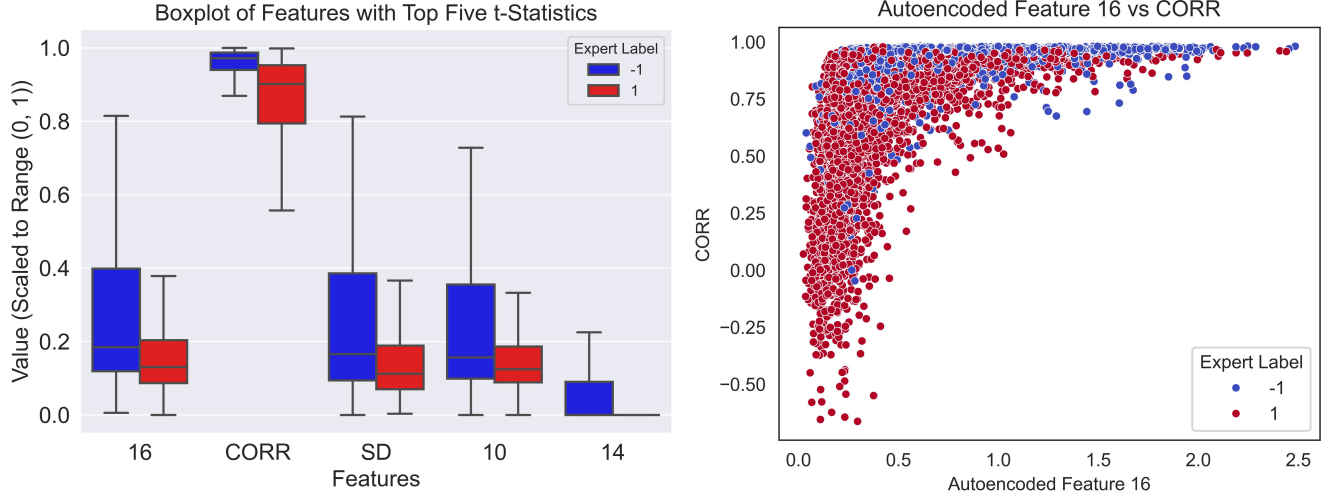


Figure 21: Image 13527 (a) Boxplot of features with highest absolute values of t statistics (b) Scatterplot of feature CORR and autoencoded feature 16

5.6 Three-model ensemble

We briefly explored a three-model ensemble where we combine the soft probabilities from our three models (neural network, Random Forest, SVM) to see if they can complement each other and lead to better performance. We explored three ways to aggregate these values: 1) product of probabilities ($Pr_{nn} \times Pr_{rf} \times Pr_{svm}$), 2) sum of pairwise products ($Pr_{nn} \times Pr_{rf} + Pr_{nn} \times Pr_{svm} + Pr_{rf} \times Pr_{svm}$), and 3) sum of probabilities ($Pr_{nn} + Pr_{rf} + Pr_{svm}$). The first two aggregation methods (product, and sum of pairwise products) yielded our highest AUROC values so far (0.9812 and 0.9811, respectively), while the sum method (0.9804) was slightly inferior to the AUROC of Random Forest (0.9807). However, the gain was modest and did not appear meaningful when inspecting the resulting classifications. The reason for this appears to be correlated errors. All three models tended to make similar mistakes. The most common mistake was misclassification of the top left clear island in image 13257. It should be noted that this manual annotation appears suspect, which may explain why all three models struggle to classify it as clear and not cloud.

5.7 Further Stability Analyses

To further analyze the stability of our modeling approach, we introduce several perturbations to the cross-validation pipeline and investigate their effects on downstream Random Forest performance. First, we train our model with the same hyperparameters but on different permutations of train and validation images.

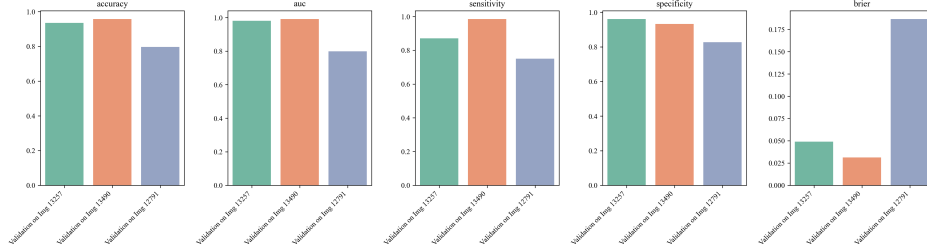


Figure 22: Performance of Random Forest on different Training/Validation Splits

We notice that the RF model continues to perform well broadly, though results for the third validation image (*Image12971*) are slightly poorer. This maybe due to a rough spatial similarity between the training images used here, affecting generalizability. However, on plotting the predicted labels, we see that our model still maintains reasonable performance.

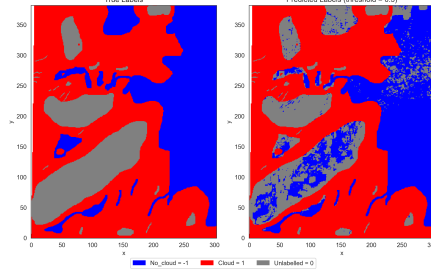


Figure 23: Predictions of RF Validated on Image 12971.

To further test the generalizability of our prediction algorithms to unseen data, we evaluate visually the predictions of all three RF models discussed above on unseen, unlabelled data.

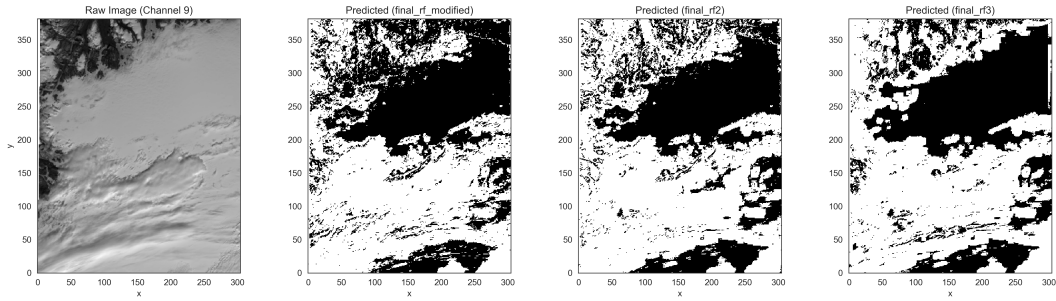


Figure 24: Predictions of RF Algorithms on Image 45178 (Unseen).

We notice that though slightly different, the models still capture very similar regions that seem plausible. More disturbed regions are correctly classified as cloudy, while smoother regions are classified as clear. This consistency across predictions is true for other unlabelled images as well. This strongly supports the robustness of our approach.

Finally, we also introduce perturbations to our cross validation pipeline, and use smaller patches, (5×5) and (2×2) , as well as removing the 2-pixel padding in our train/test split to analyze the effects on hyperparameter selection, and the performance of our algorithm on test (not validation) data. We note that these lead to different but very similar hyperparameters, and performance on test data stays similar.

5.8 Conclusions

We thus show stable, robust modelling for distinguishing "Clear" pixels from "Cloudy" pixels. Our results show that non-linear models such as Random Forests and Neural Networks perform better than linear models such as SVMs. These models also converge and are stable in their predictions. We demonstrate empirically the effectiveness of our engineered features, as well as the latent features from the Autoencoder. Finally, we conduct Post-Hoc EDA to better understand our predictions and their shortcomings, and a stability analysis to show that our results are robust, and our models are capable of capturing real, general patterns effectively.

Such predictive models could be very useful, particularly in expert-in-the-loop settings, where supervision and fine-tuning could yield trustworthy labels for cloudy regions. This would help automate and understand imaging results from the MISR satellite in context, and reduce bias from further investigations and results using these images.

6 Bibliography

References

- [1] T. Shi, B. Yu, E. E. Clothiaux, and A. J. Braverman, “Daytime arctic cloud detection based on multi-angle satellite data with case studies,” *J. Am. Stat. Assoc.*, vol. 103, no. 482, pp. 584–593, Jun. 2008.
- [2] N. Kanopoulos, N. Vasanthavada, and R. L. Baker, “Design of an image edge detection filter using the sobel operator,” *IEEE J. Solid-State Circuits*, vol. 23, no. 2, pp. 358–367, Apr. 1988.
- [3] T. E. A. Z. R. Q., *Support vector machines*, <https://teazrq.github.io/SMLR/support-vector-machines.html>, Accessed: 2025-11-18.

A Academic honesty Statement

The work was done by me with occasional help from online resources which we cited in the coding part and/or the report. I think academic honesty is important in academics. Fabricating data is detrimental to the scientific community. Using the work of other people as your own is harmful to other researchers.

A.1 Collaborators

Lucas Zhou joined group meetings twice but did not contribute to any coding and writing work.

A.2 LLM Usage

Coding

We used ChatGPT to debug our code, make and fix plots to improve aesthetics, but did not generate code from scratch in the following sections: 1. Debug code for various plots of validation images in posthoc EDA

Writing

We used ChatGPT to polish sentences and improve sentence flow, as well as debugging written code on Overleaf in the following sections: 1. Code for arranging plots and adding equations in Section 5.1 2. Creating a template Docstring for the `SpatialAutoencoder` class, which then modified.